

Chapter 20

“Digital Circuits”

Credit – name: “desc” (need permission)

As powerful as the *mark* notation is, and as brilliant as its ability to derive logic from first principles is (unique among formal systems as far as I know), it does suffer from a few limitations. The first is cultural; it is not a well-known formalism. Second, it is not a typographic system; while ideal for paper and pencil, it doesn’t translate well to modern editing tools. Third, and most grievous, it cannot represent all possible self-referential forms and thus cannot be used to determine their total number or create a complete taxonomical characterization of them.

Therefore, over the next few chapters, we are going to attack the foundations of logic from a different perspective. Over the next five chapters, five different tools for analyzing self-referential logical forms are going to be developed. These five tools work well together and as a set both reinforce each other and address the limitations of the mark notation.

Motivation

Before we begin it is worth reviewing why we are going to so much trouble. The impetus is the conflict between relativity and quantum nonlocality. Nonlocality implies FTL, which relativity says is impossible because FTL implies time travel. Given spacelike causality one gets both. The strongest argument against time travel is the Grandfather paradox, the possibility of a causal paradox. The reason for pursuing imaginary truthvalues is that they resolve the problem of logical paradoxes by introducing conjugate bases for logic. QTP is simply the hypothesis that conjugate bases might resolve causal paradoxes in physics and do so in a way that solves the measurement problem. Put another way, the Universe allows time travel (at the quantum level, not at the classical level), but it comes with a cosmic censor that prohibits temporal paradox. The cosmic censor *is* the measurement process; an observer independent objective process that collapses a quantum system in the one-and-only one basis in which it is not paradoxical.

Road Map

Five things are a lot to learn. Particularly when knowing four of them, any four, helps in understanding the remaining one. Unfortunately, they cannot be presented simultaneously, but must be presented sequentially, one at-a-time. So, to make the pedagogy as effective as possible, here is a road map of the five tools, along with brief notes on what they bring to the party.

The Five Tools

1. Digital Circuit Schematics – how computer engineers represent logic and self-reference (effortlessly), a **graphical** idiom based on logic gates that make self-reference obvious.
2. Truth Tables – a **table** driven look at how inputs become outputs, showing how to combine gates to specify the operators that will be needed.

3. Successor Vectors – a **vector** look at truth tables which allows for representing them numerically; a prerequisite for the abstraction of attractor structures.
4. Attractor Structures – a **pictorial** representation which shows fixed points while clarify multi-valuedness and phase; all critical for concepts of equality.
5. NonLinear Logic (NLL) – a formal **typographic** system of logic; a symbolic basis for proofs and theorems, which surpasses the limitations of the *mark*.

Each of these tools enable the determination of the total number of logical forms, both linear (combinatorial) and self-referential (sequential). Furthermore, there is a one-to-one correspondence between all five tools. They are different ways of looking at the same problem space. They support each other and inform each other. Most significantly, they all agree on the total number of possible logical forms; strong evidence that we are looking at the problem the right way.

Digital Circuits

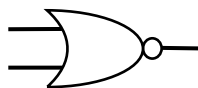
We'll only be concerned with *schematics* of digital circuits; their actual electrical instantiation will not be needed. However, as a historical aside, early in this research project some effort was expended physically implementing self-referential digital circuits with modern hardware. Within the electrical limits of the technology, the predictions of the formal system to be discussed here were confirmed. Suffice it to say it was a cool period of research while I served as an adjunct professor at Sacramento State University.

The central idiom in digital circuits is the logic *gate*. It has one or two inputs and just one output. Schematically, a gate is represented as a closed figure. Typically, the inputs are on the left and the output is on the right. Gates are connected to each other by simple lines, representing the electrical connections on, say a circuit board. An input, but more often an output, may have a small circle appended to the closed figure, which means any signal will be inverted. The notation for the logical values has three typical forms; the logician will use true (T) and false (F), the digital engineer will use 1 and 0, and the electrical engineer will use high (typically +5 volts) and low (typically 0 volts). Just to confuse things even further, the +5 volts may be called the source (or, get this, Vcc) and the 0 volts will be called the sink or ground. There is no shortage of standards.

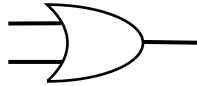
Each line connecting gates (or connecting to the outside world) is treated as a variable and given a label, typically a single character such as 'A' or 'B'. Each gate is also given a name, such as AND or NOR, and is usually highlighted in SMALL CAPS. Each gate will also be given a symbol, such as '^' for AND, or '~' for NOT. Again, there is no shortage of standards, but all these various notations allow for very concise, accurate, and error free representation of intent. They become second nature pretty quick.

For those new to all this there are only 16 possible one and two input gates. Surprisingly, there are actually only 4 closed figures used to indicate these 16 possibilities, the differences all come down to where those pesky little inversion circles show up on the closed figures.

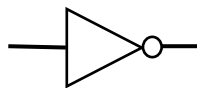
Below is the NOR gate



It is most like the *mark* notation in Laws of Form. Note the small circle on the output to indicated inversion. An OR gate ($\vee$) by contrast, has the same shape, but is missing the circle



A NOT gate ($\sim$) looks like this



which is also like the *mark* of the form, just with a single input rather than the dual input of the NOR gate or even more.

At the end of this chapter, all 16 gates will be presented in a single table. But before committing to that, a preview of the next tool is in order. This is because the next tool comes in two flavors.

Truth Tables – Gates Only

The job of the truth table is to map between all the possible inputs and what specific output each permutation of the inputs creates. Here is the truth table for the OR gate.

A	B	OR
F	F	F
F	T	T
T	F	T
T	T	T

This is a formal definition of what the word *or* more or less means in conventional usage, but in this case its either A or B or both.

For the NOR gate.

A	B	NOR
F	F	T
F	T	F
T	F	F
T	T	F

Note how the input columns between these two tables are the same, but the output column (column 3) is inverted with respect to the OR gate. If you wish to correlate with the *mark* notation, T is m (the marked state), and F is n (the unmarked state).

The truth table for the NOT gate is even simpler.

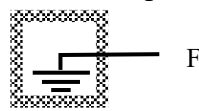
A	NOT
F	T
T	F

Based on two input tables, the output column cell for each row has just two possible values, T or F. There are therefore $2^4 = 16$ possible truth tables with two inputs. Thus, we need 16 gates. If you are wondering how the single input gates fold into the dual input gates, you are ahead of the class.

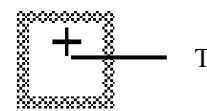
Table of Logic Gates

Below is the table of 16 gates, with symbols, gate names and logical names. The truth tables have been paired to save space; gives them two output columns – it's just a shorthand.

A	B	F	T
F	F	F	T
F	T	F	T
T	F	F	T
T	T	F	T

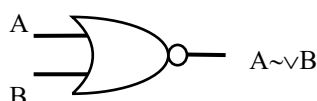


SINK
(Contradiction)

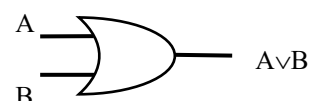


SOURCE
(Tautology)

A	B	$A \sim \vee B$	$A \vee B$
F	F	T	F
F	T	F	T
T	F	F	T
T	T	F	T

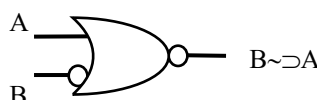


NOR
(Joint Denial)

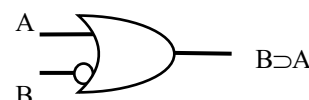


OR
(Disjunction)

A	B	$B \sim \supset A$	$B \supset A$
F	F	F	T
F	T	T	F
T	F	F	T
T	T	F	T

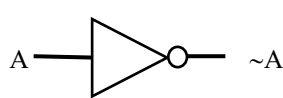


DISIMPLIES
(Not if B then A)

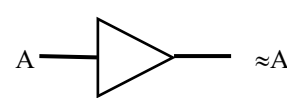


IMPLIES
(If B then A)

A	B	$\sim A$	$\approx A$
F	F	T	F
F	T	T	F
T	F	F	T
T	T	F	T

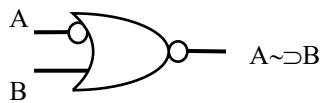


NOT
(Negation)

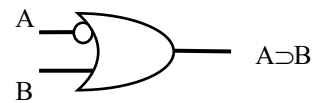


BUF
(Proposition)

A	B	$A \sim \supset B$	$A \supset B$
F	F	F	T
F	T	F	T
T	F	T	F
T	T	F	T

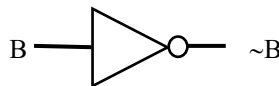


DISIMPLIES
(Not if A then B)

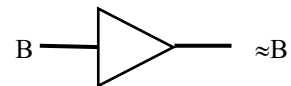


IMPLIES
(If A then B)

A	B	$\sim B$	$\approx B$
F	F	T	F
F	T	F	T
T	F	T	F
T	T	F	T

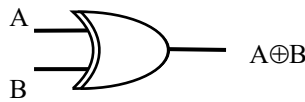


NOT
(Negation)

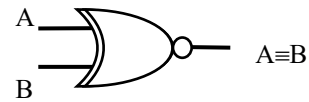


BUF
(Proposition)

A	B	$A \oplus B$	$A \equiv B$
F	F	F	T
F	T	T	F
T	F	T	F
T	T	F	T

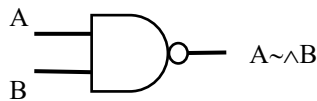


XOR
(Exclusive Or)

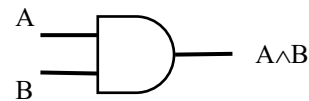


COMP
(Bicomparator)

A	B	$A \sim \wedge B$	$A \wedge B$
F	F	T	F
F	T	T	F
T	F	T	F
T	T	F	T



NAND
(Alternative Denial)



AND
(Conjunction)

You might be wondering why truth tables have been introduced in this chapter when they will be covered in the next one. Two reasons; we really need them to make sense of the digital circuit gates, and in this form, they only apply to what are called combinatorial circuits. A combinatorial circuit has no feedback, no self-reference; the inputs are independent of the outputs. In sequential circuits, there is feedback; outputs can feed back to inputs and thus are self-referential. In those cases, both truth tables *input* and *output* columns can apply to the same variable.